06

Testes Automatizados e TDD Avançado

Testes Automatizados e TDD Avançado

Como ganhar confiança para alterar Laravel, Inertia e React.

Roteiro da aula

- Problema real no projeto base
- PHPUnit
- Feature test
- Unit test
- Mocking e Faking
- React + Testing Library
- Parallel Testing
- Snapshot Testing
- Projeto do módulo

Problema real no projeto base

O time precisa alterar regras de agendamento e da API sem medo de quebrar fluxos existentes.

Pergunta

Como saber se a criação de agendamento ainda funciona depois de refatorar?

PHPUnit

PHPUnit organiza testes em classes com métodos e asserções explícitas.

Exemplo

Exemplo de código

```
$appointment = Appointment::factory()->create([
    'scheduled_at' => '2026-10-20 09:00:00',
]);

$this->assertModelExists($appointment);
$this->assertSame(
    '09:00',
    $appointment->scheduled_at->format('H:i'),
);
```

Feature test

Pergunta
"O usuário consegue executar o fluxo?"

Exemplo

Exemplo de código

```
$this->get('/agenda?date=2026-10-15')
    ->assertOk()
    ->assertInertia(fn (Assert $page) => $page
        ->component('Appointments/Index')
        ->has('appointments', 1)
        ->where('appointments.0.patientName', 'Maria')
    );
```

Unit test

Pergunta
"Essa regra isolada funciona?"

Exemplo

Exemplo de código

```
$repository->shouldReceive('hasConflict')
    ->once()->andReturnTrue();

$action = new CreateAppointmentAction($repository);

$this->expectException(TimeSlotUnavailable::class);
$action->execute($payload);
```

Mocking e Faking

Fake
Substitui um serviço externo por uma versão controlada.

Mock
Verifica se uma colaboração aconteceu.

Exemplo

Exemplo de código

```
Fake HTTP
Http::fake(['calendar.example/*' =>
    Http::response(['available' => true])
]);

Mock Laravel com Mockery
$repository = Mockery::mock(AppointmentRepository::class);
$repository->shouldReceive('hasConflict')
    ->once()->andReturn(false);

Mock no frontend com Vitest
vi.mock('@inertiajs/react', () => ({
    router: { get: vi.fn() },
}));

expect(router.get).toHaveBeenCalledWith(
    '/agenda', expect.any(Object), expect.any(Object)
);
```

React + Testing Library

Renderização

Valida dados recebidos pelo componente.

Estado vazio

Confirma a mensagem sem agendamentos.

Interação

Simula a seleção de filtros pelo usuário.

Exemplo de código

```
render(<AppointmentList appointments={[appointment]} />);
expect(screen.getByText('Mariana Alves')).toBeInTheDocument();

render(<AppointmentList appointments={[]} />);
expect(screen.getByText('Nenhuma consulta para os filtros escolhidos.'))
    .toBeInTheDocument();

await user.selectOptions(screen.getByLabelText('Status'), 'confirmed');
await user.click(screen.getByRole('button', { name: 'Atualizar agenda' }));
expect(router.get).toHaveBeenCalled();
```

Parallel Testing

Problema
Uma suíte grande demora.

Solução
Executar testes em paralelo.

Cuidado
Testes precisam isolar banco, cache, arquivos e dependências externas.

Exemplo de código

```
php artisan test --parallel

php artisan test --parallel --processes=4
```

Snapshot Testing com PHPUnit

Uso
Comparar a resposta JSON com um arquivo esperado e versionado.
Exemplo
Cuidado
Atualize o snapshot somente depois de revisar a mudança.

Exemplo de código

```
$this->assertJsonStringEqualsJsonFile(
    base_path('tests/__snapshots__/agenda.json'),
    $response->getContent(),
);
```

Projeto do módulo

Missão
Criar uma suíte de testes para os fluxos principais do sistema guia.

Entregas
- teste de criação de agendamento;
- teste de conflito de horário;
- fake para API externa;
- teste de componente React;
- execução paralela configurada;
- snapshot de resposta da API de agenda.

Checklist final

- O aluno deve conseguir
- escrever testes PHPUnit;
- diferenciar unit e feature test;
- testar componentes React;
- aplicar mocks com critério;
- rodar testes em paralelo;
- criar snapshots sem esconder regras importantes.
-